

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

—o0o—



BIG PROJECT 1 REPORT

Floating Point Adder and Subtractor

SUPERVISOR: Nguyễn Trung Hiếu

SUBJECT: Digital System Design
and Verification

GROUP: 08

List of Members

STT	MSSV	Họ Và Tên	Lớp
1	2210780	Nguyễn Đại Đồng	L01
2	2213874	Nguyễn Thanh Tùng	L01
3	2213496	Nguyễn Quốc Tín	L01

Ho Chi Minh, ../../20..

Mục lục

1	Requirement	1
2	Design FPU	2
2.1	Bộ tiền xử lý liên quan đến Exponent	3
2.2	Bộ tiền xử lý liên quan đến Mantissa	5
2.3	Bộ xử lý dấu của phép tính	7
2.4	Bộ xử lý các trường hợp đặc biệt ở ngõ vào	9
2.5	Bộ căn chỉnh Exponent	12
2.6	Bộ cộng trừ Mantissa	13
2.7	Bộ chuẩn hóa Mantissa	14
2.8	Bộ tìm vị trí bit 1 của MANTISSA sau khi chuẩn hóa	15
2.9	Bộ làm tròn cho giá trị Mantissa và Exponent	15
3	Simulation and Verification	17
4	Implement on FPGA	21
5	Synthesis my design	23

Danh sách hình vẽ

2.1	Sơ đồ khối tổng quan của bộ FPU.	2
2.2	Bộ tính toán giá trị $ EXP_A - EXP_B $	3
2.3	Bìa K của bộ so sánh 4-bit (COMP_4bit).	4
2.4	Bộ so sánh 28-bit sử dụng các so sánh lan truyền.	6
2.5	Bộ so sánh 28-bit sử dụng phép trừ sử dụng giải thuật của bộ CLA.	6
2.6	Rút gọn bìa K theo bảng sự thật 2.1.	8

4.1	RTL Viewer của bộ FPU.	21
4.2	Kết quả khi thực hiện trên kit DE2.	22

Danh sách bảng

1.1	Input and Output Ports.	1
2.1	Bảng sự thật của <code>SIGN_RESULT</code>	8
2.2	Các giá trị đặc biệt ở ngõ vào.	9
2.3	Bảng sự thật của bộ phát hiện các giá trị đặc biệt.	11

List of Listings

1	Rút gọn từ Bìa K của bộ so sánh 4-bit (<code>COMP_4bit</code>).	4
2	Hàm golden model để thực hiện test.	17
3	Hàm tính sai số giữa kết quả của DUT và kết quả của golden model để thực hiện test.	17
4	Hàm Testbench chính.	18
5	Kết quả của Testbench.	19
6	<code>genus_synthesis.tcl</code>	23
7	File <code>constraint.sdc</code>	23

Chapter 1. Requirement

Thiết kế một bộ **Floating Point (FPU)** có thể cộng và trừ hai số **floating-point 32-bit**. Phương pháp thiết kế sử dụng thuần là bộ tổ hợp.

Sau khi thiết kế thành công, cần:

- Viết chương trình kiểm định thiết kế **FPU** trên.
- Thực hiện thiết kế trên KIT FPGA để kiểm tra chức năng của thiết kế.
- Tiến hành tổng hợp thiết kế sử dụng các standard cell trong thư viện có sẵn tại các trường hợp thực tế và loại bỏ các glitches.

Ngõ vào và ngõ ra của thiết kế được mô tả như sau:

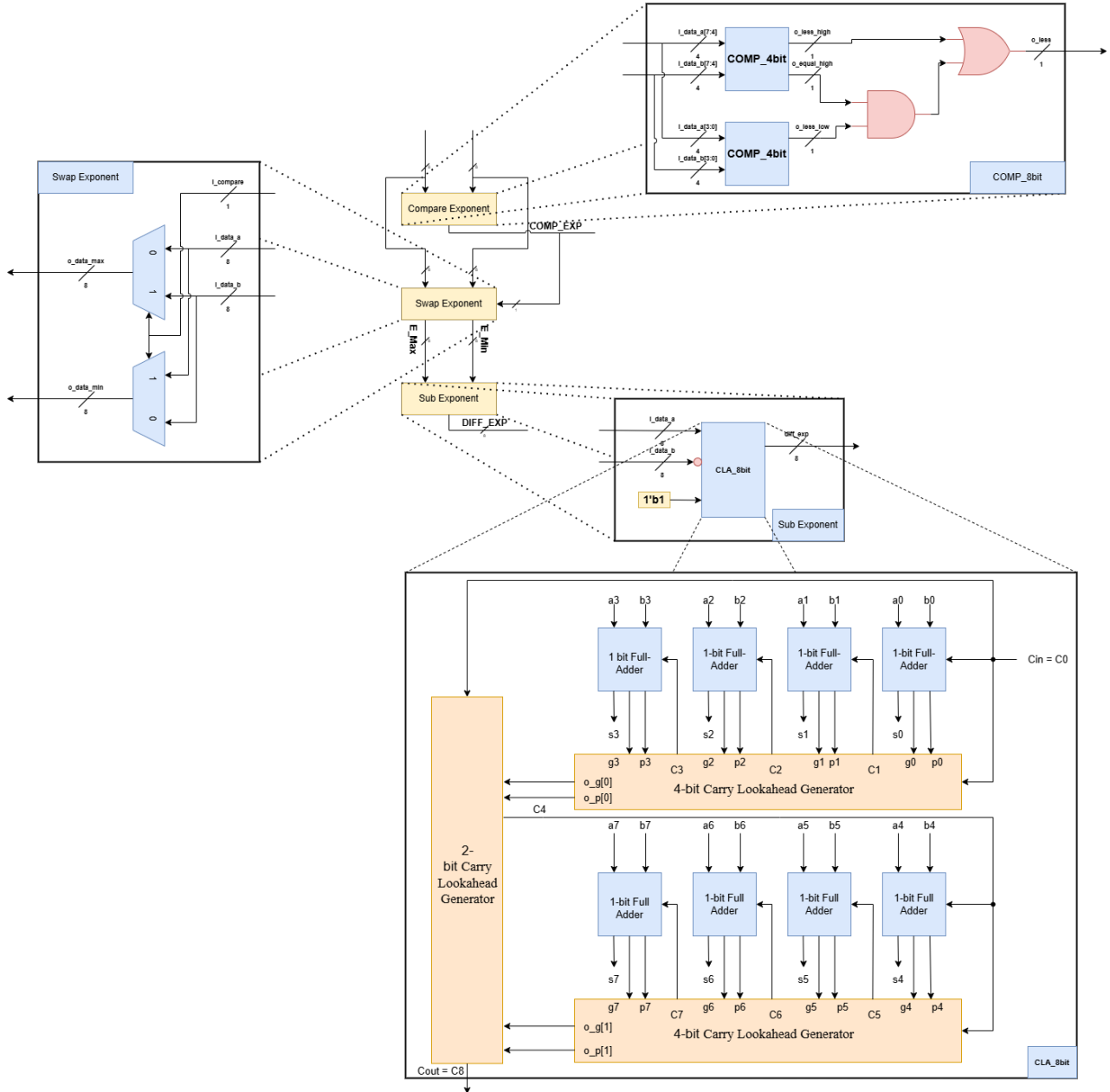
Name	Port Type	Width	Description
i_32_a	Input	32	32-bit floating point number A
i_32_b	Input	32	32-bit floating point number B
i_add_sub	Input	1	Operation: 0 = add, 1 = sub
o_32_s	Output	32	32-bit floating point output
o_ov_flag	Output	1	Overflow Flag: 1 = overflow, 0 = no overflow occurs
o_un_flgag	Output	1	Underflow Flag: 1 = underflow, 0 = no underflow occurs

Bảng 1.1: Input and Output Ports.

Page 2 | 26

Hình 2.1: Sơ đồ khối tổng quan của bộ FPU.

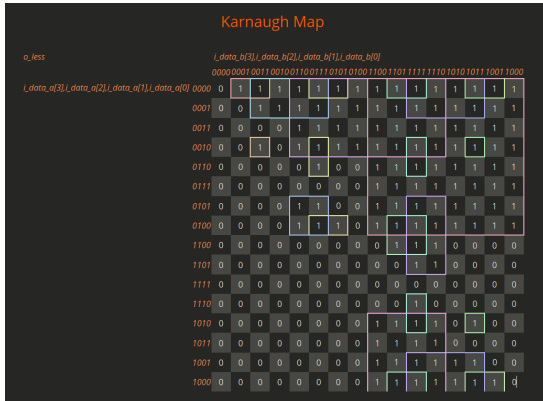
2.1 Bộ tiền xử lý liên quan đến Exponent



Hình 2.2: Bộ tính toán giá trị $|EXP_A - EXP_B|$.

Bộ có chức năng thực hiện tính giá trị trị khác nhau giữa hai Exponent của A và B. Trong đó,

- Bộ **COMP_4bit**: là bộ so sánh 4-bit hai số được triển khai dựa vào bảng sự thật và rút gọn bìa K như sau:



Hình 2.3: Bìa K của bộ so sánh 4-bit (COMP_4bit).

```

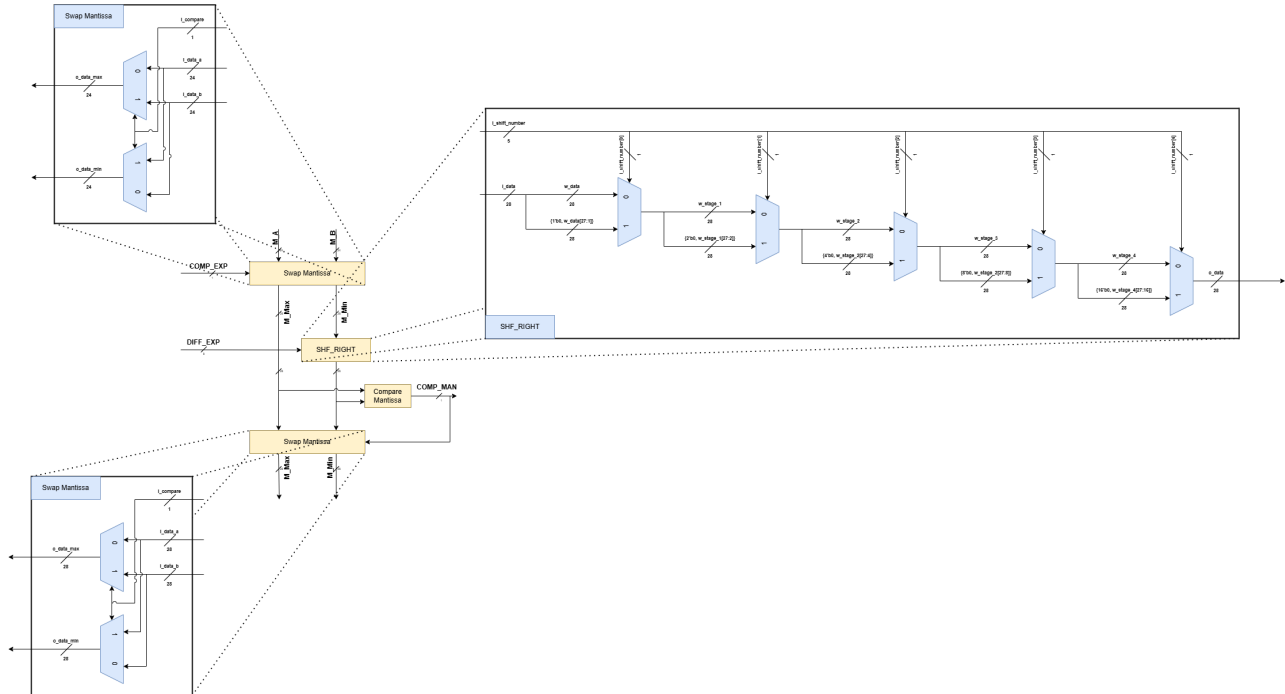
1      assign o_less = (~i_data_a[3] & ~
      i_data_a[2] & ~i_data_a[1] & ~i_data_a[0]
      & i_data_b[0]) | (~i_data_a[3] & ~
      i_data_a[2] & ~i_data_a[1] & i_data_b[1])
      | (~i_data_a[3] & ~i_data_a[2] &
      i_data_b[2]) | (~i_data_a[3] & i_data_b
      [3]) | (~i_data_a[3] & ~i_data_a[2] & ~
      i_data_a[0] & i_data_b[1] & i_data_b[0])
      | (~i_data_a[3] & ~i_data_a[1] & ~
      i_data_a[0] & i_data_b[2] & i_data_b[0])
      | (~i_data_a[3] & ~i_data_a[1] & i_data_b
      [2] & i_data_b[1]) | (~i_data_a[3] & ~
      i_data_a[0] & i_data_b[2] & i_data_b[1] &
      i_data_b[0]) | (~i_data_a[2] & ~i_data_a
      [1] & ~i_data_a[0] & i_data_b[3] &
      i_data_b[0]) | (~i_data_a[2] & ~i_data_a
      [1] & i_data_b[3] & i_data_b[1]) | (~
      i_data_a[2] & i_data_b[3] & i_data_b[2])
      | (~i_data_a[2] & ~i_data_a[0] & i_data_b
      [3] & i_data_b[1] & i_data_b[0]) | (~
      i_data_a[1] & ~i_data_a[0] & i_data_b[3]
      & i_data_b[2] & i_data_b[0]) | (~i_data_a
      [1] & i_data_b[3] & i_data_b[2] &
      i_data_b[1]) | (~i_data_a[0] & i_data_b
      [3] & i_data_b[2] & i_data_b[1] &
      i_data_b[0]);

2
3      assign o_equal = ~(i_data_a ^
      i_data_b);
4

```

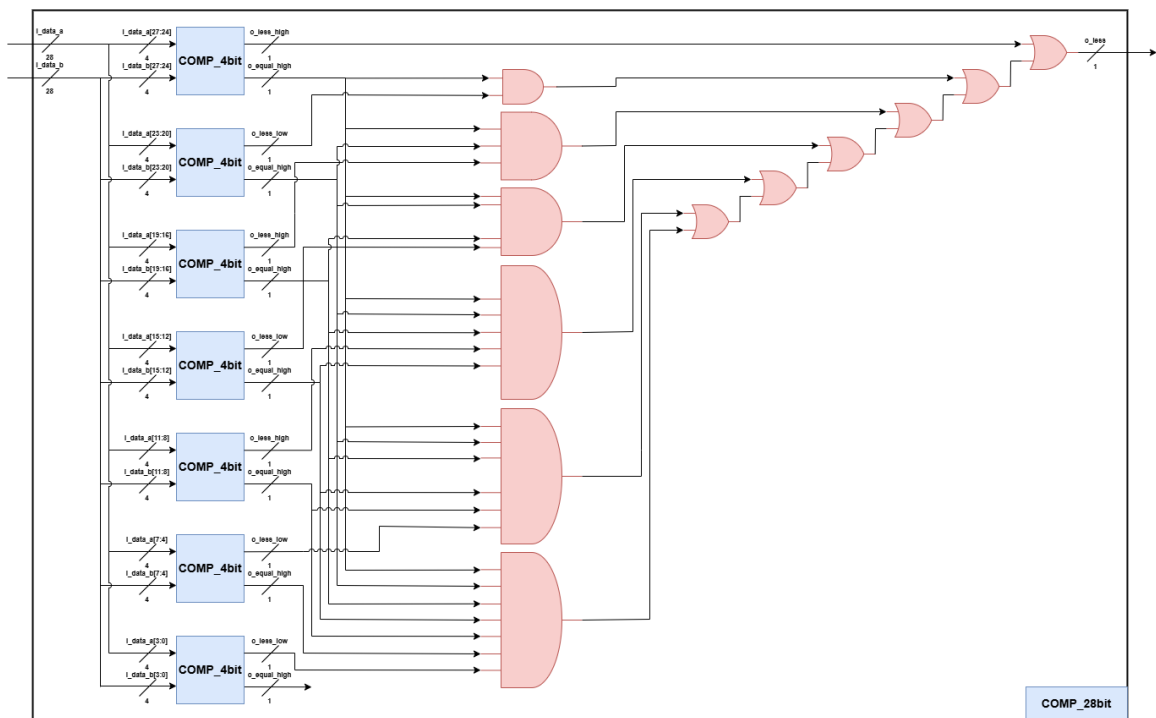
Listing 1: Rút gọn từ Bìa K của bộ so sánh 4-bit (COMP_4bit).

2.2 Bộ tiền xử lý liên quan đến Mantissa



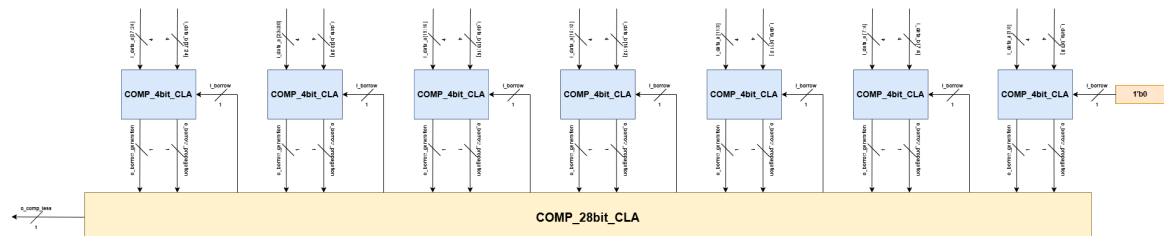
Bộ Compare Mantissa, nhóm em đề xuất hai phương pháp như sau:

Phương pháp 1. Sử dụng phương pháp xây dựng từ bộ COMP_4bit để tạo nên bộ COMP_28bit như sau:



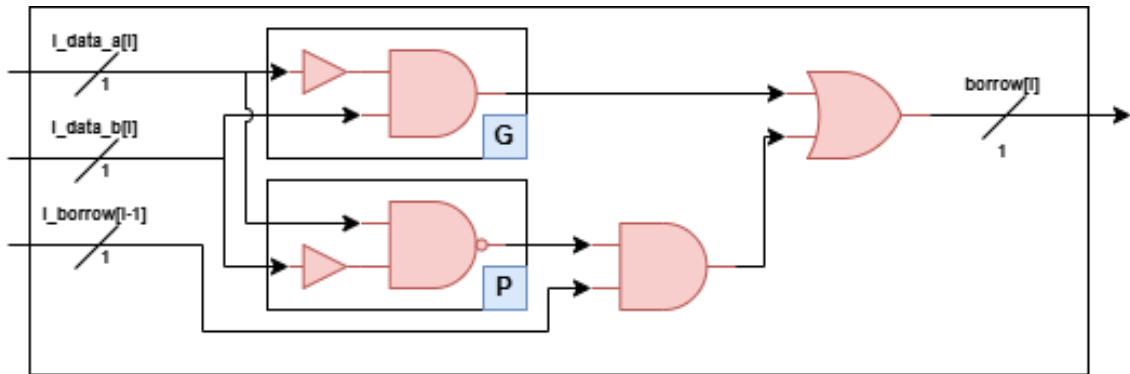
Hình 2.4: Bộ so sánh 28-bit sử dụng các so sánh lan truyền.

Phương pháp 2. Sử dụng giải thuật cộng của bộ CLA (Carry Lookahead Adder) như sau:



Hình 2.5: Bộ so sánh 28-bit sử dụng phép trừ sử dụng giải thuật của bộ CLA.

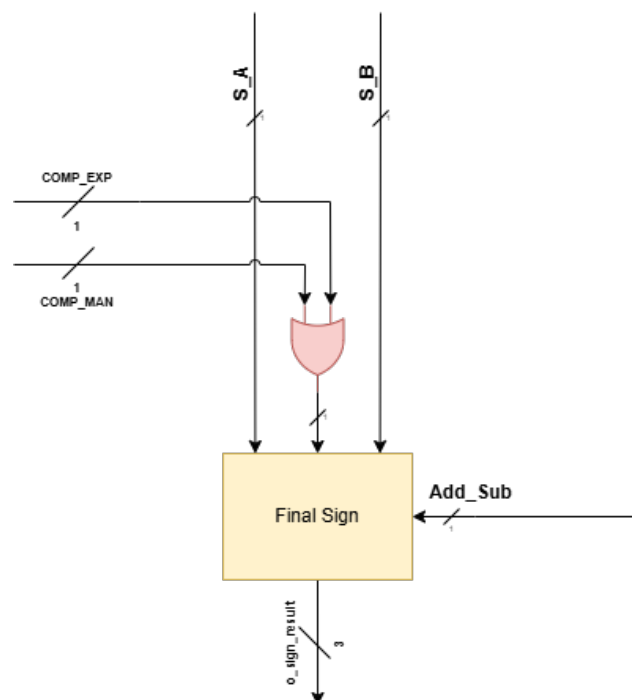
Tuy nhiên với bộ tạo ra tín hiệu P và G sẽ khác do đây là phép trừ hai bit được biểu dưới dạng như:



Và từ đó sẽ tiến hành như bộ CLA.

Từ hai phương pháp trên, nhóm em quyết định sử dụng bộ 2.4 để có thể dễ dàng triển khai bộ COMP_28bit.

2.3 Bộ xử lý dấu của phép tính



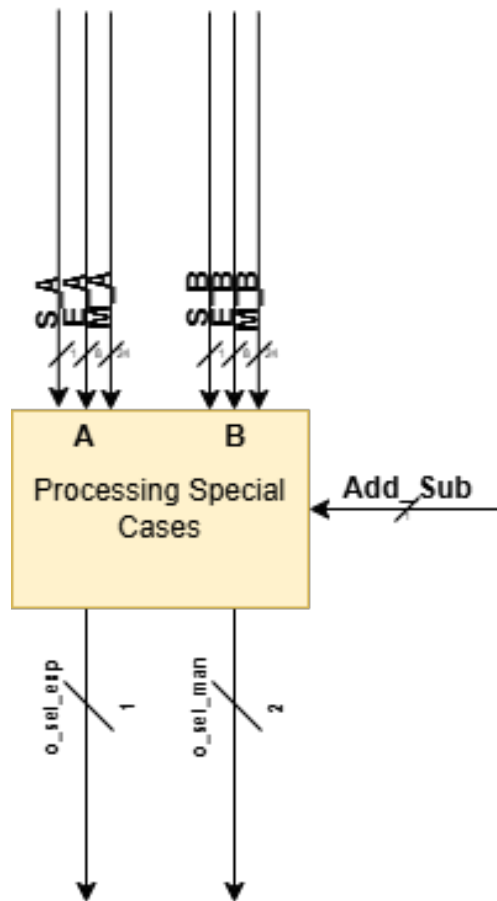
i_add_sub	i_sign_a	i_sign_b	i_comp	o_sign_result	
0	0	0	X	0	$(A + B) > 0$
0	0	1	0	0	$(A \geq B) \rightarrow (A - B) > 0$
0	0	1	1	1	$(A < B) \rightarrow (A - B) < 0$
0	1	0	0	1	$(A \geq B) \rightarrow (-A + B) < 0$
0	1	0	1	0	$(A < B) \rightarrow (-A + B) > 0$
0	1	1	X	1	$(-A - B) < 0$
1	0	0	0	0	$(A \geq B) \rightarrow (A - B) > 0$
1	0	0	1	1	$(A < B) \rightarrow (A - B) < 0$
1	0	1	X	0	$(A - -B) > 0$
1	1	0	X	1	$(-A - B) < 0$
1	1	1	0	1	$(A \geq B) \rightarrow (-A - -B) < 0$
1	1	1	1	0	$(A < B) \rightarrow (-A - -B) > 0$

Bảng 2.1: Bảng sự thật của SIGN_RESULT.



Hình 2.6: Rút gọn bìa K theo bảng sự thật 2.1.

2.4 Bộ xử lý các trường hợp đặc biệt ở ngõ vào



Bảng 2.2: Các giá trị đặc biệt ở ngõ vào.

i_add_sub	i_data_a	i_data_b	o_data_fpu
0	$+\infty$	Number	$+\infty$
0	$+\infty$	$-\infty$	NaN
0	$+\infty$	$+\infty$	$+\infty$
0	$-\infty$	Number	$-\infty$
0	$-\infty$	$-\infty$	$-\infty$

Continued on next page ...

Bảng 2.2 – continued from previous page

i_add_sub	i_data_a	i_data_b	o_data_fpu
0	$-\infty$	$+\infty$	NaN
0	NaN	X	NaN
0	X	NaN	NaN
1	$+\infty$	Number	$+\infty$
1	Number	$+\infty$	$-\infty$
1	$+\infty$	$-\infty$	$+\infty$
1	$+\infty$	$+\infty$	NaN
1	$-\infty$	Number	$-\infty$
1	Number	$-\infty$	$+\infty$
1	$-\infty$	$-\infty$	NaN
1	$-\infty$	$+\infty$	$-\infty$
1	NaN	X	NaN
1	X	NaN	NaN

Với các giá trị đặc biệt ở ngõ vào có đặc tính đặc biệt như sau:

- Đối với các số liên quan đến $\pm\infty$ thì có
 - Exponent: $E = 8'hFF$
 - Mantissa: $M = 24'h00$
- Đối với các số liên quan đến NaN thì có
 - Exponent: $E = 8'hFF$
 - Mantissa: $M \neq 24'h00$

Từ đó ta tạo ra hai giá trị:

$$+ \mathbf{E} = \&i_Exponent$$

$$+ \mathbf{M} = \sim |(i_Mantissa)$$

Bảng 2.3: Bảng sự thật của bộ phát hiện các giá trị đặc biệt.

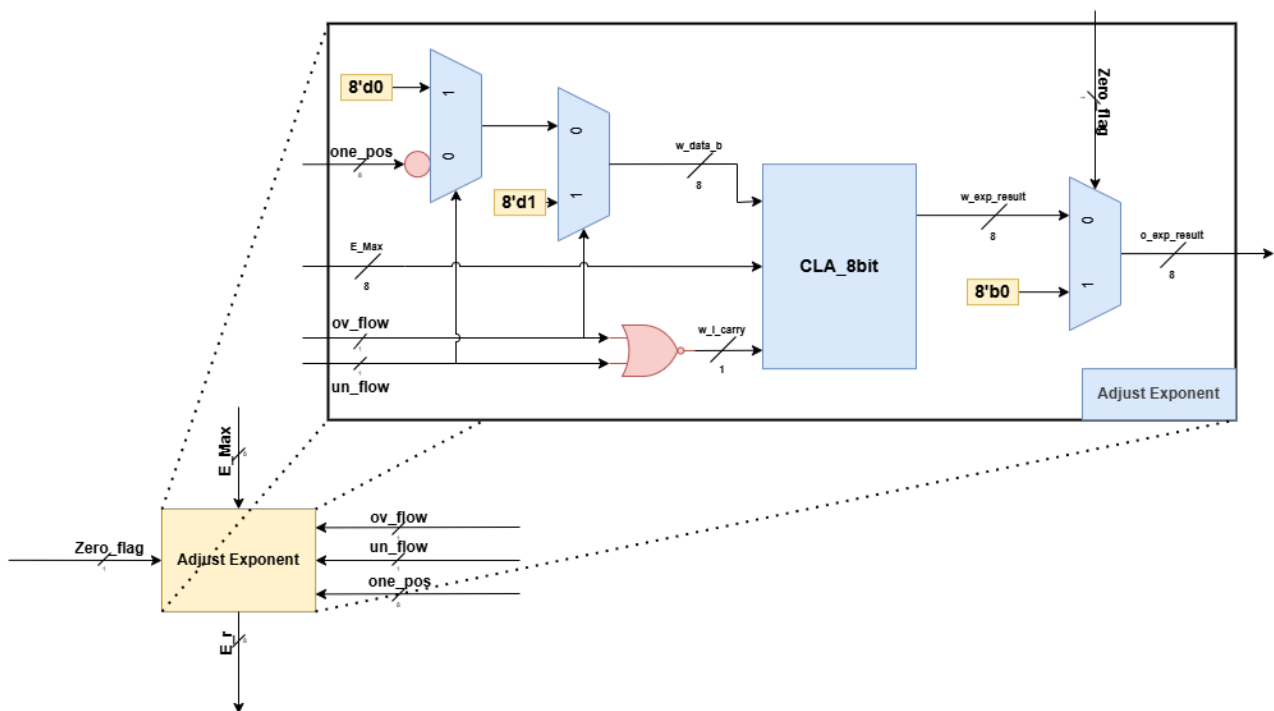
i_add_sub	A			B			S				
	S	E	M	S	E	M	S	E	M	S_M_1	S_M_0
0	0	1	1	X	0	X	0	1	1	1	0
0	0	1	1	1	1	1	0	1	0	1	1
0	0	1	1	0	1	1	0	1	1	1	0
0	1	1	1	X	0	X	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	0	1	1	1	1	0	1	1
0	X	1	0	X	X	X	X	1	0	1	1
0	X	X	X	X	1	0	X	1	0	1	1
1	0	1	1	X	0	X	0	1	1	1	0
1	X	0	X	0	1	1	1	1	1	1	0
1	0	1	1	1	1	1	0	1	1	1	0
1	0	1	1	0	1	1	X	1	0	1	1
1	1	1	1	X	0	X	1	1	1	1	0
1	X	0	X	1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	X	1	0	1	1
1	1	1	1	0	1	1	1	1	1	1	0
1	X	1	0	X	X	X	X	1	0	1	1

Continued on next page ...

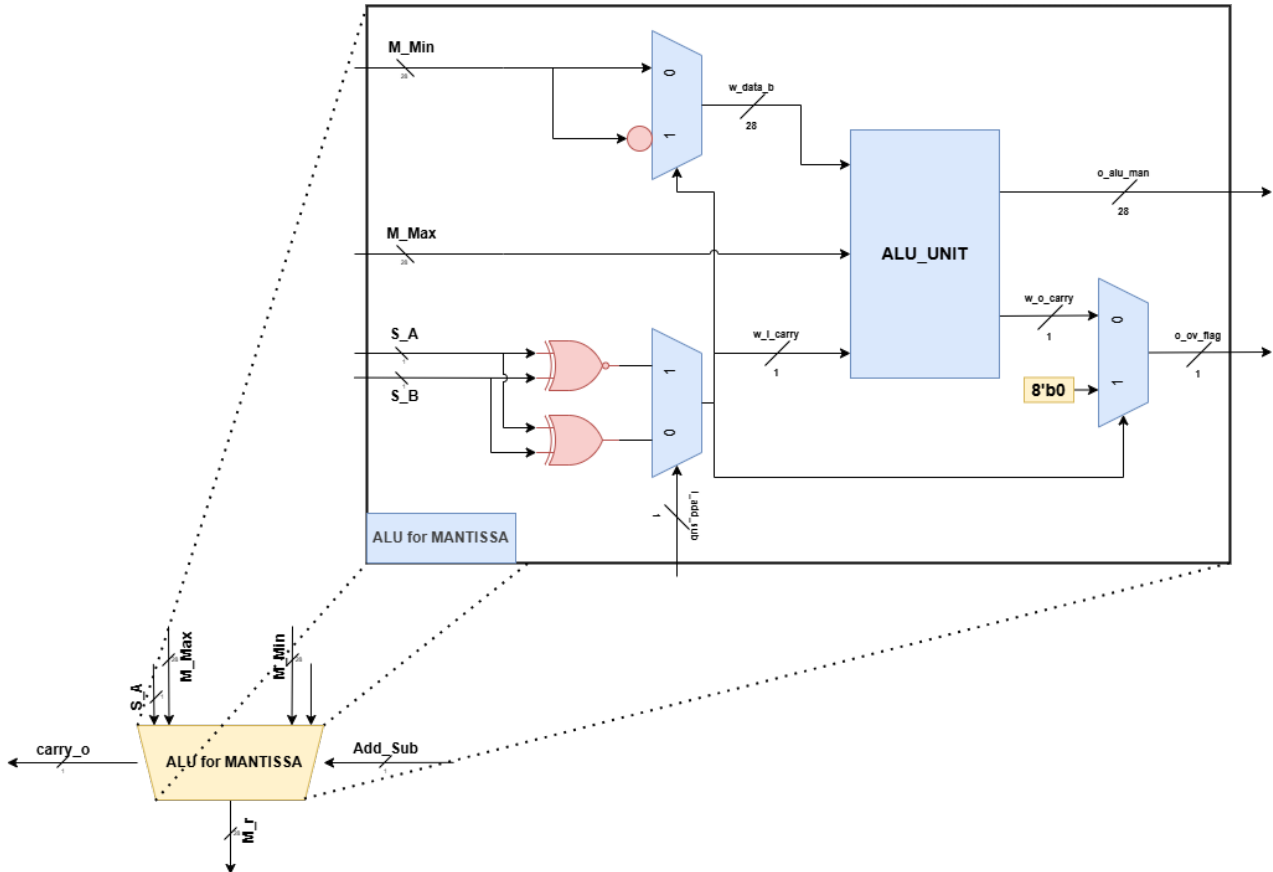
Bảng 2.3 – continued from previous page

1	X	X	X	X	1	0	X	1	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---

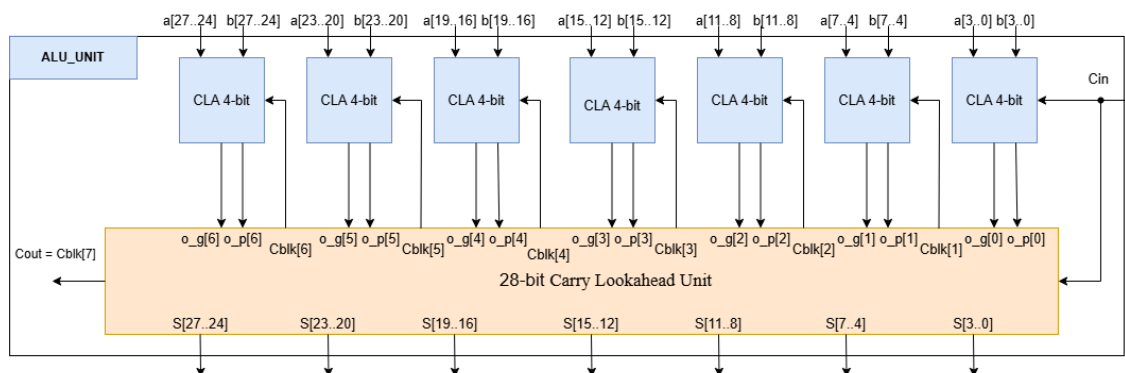
2.5 Bộ căn chỉnh Exponent



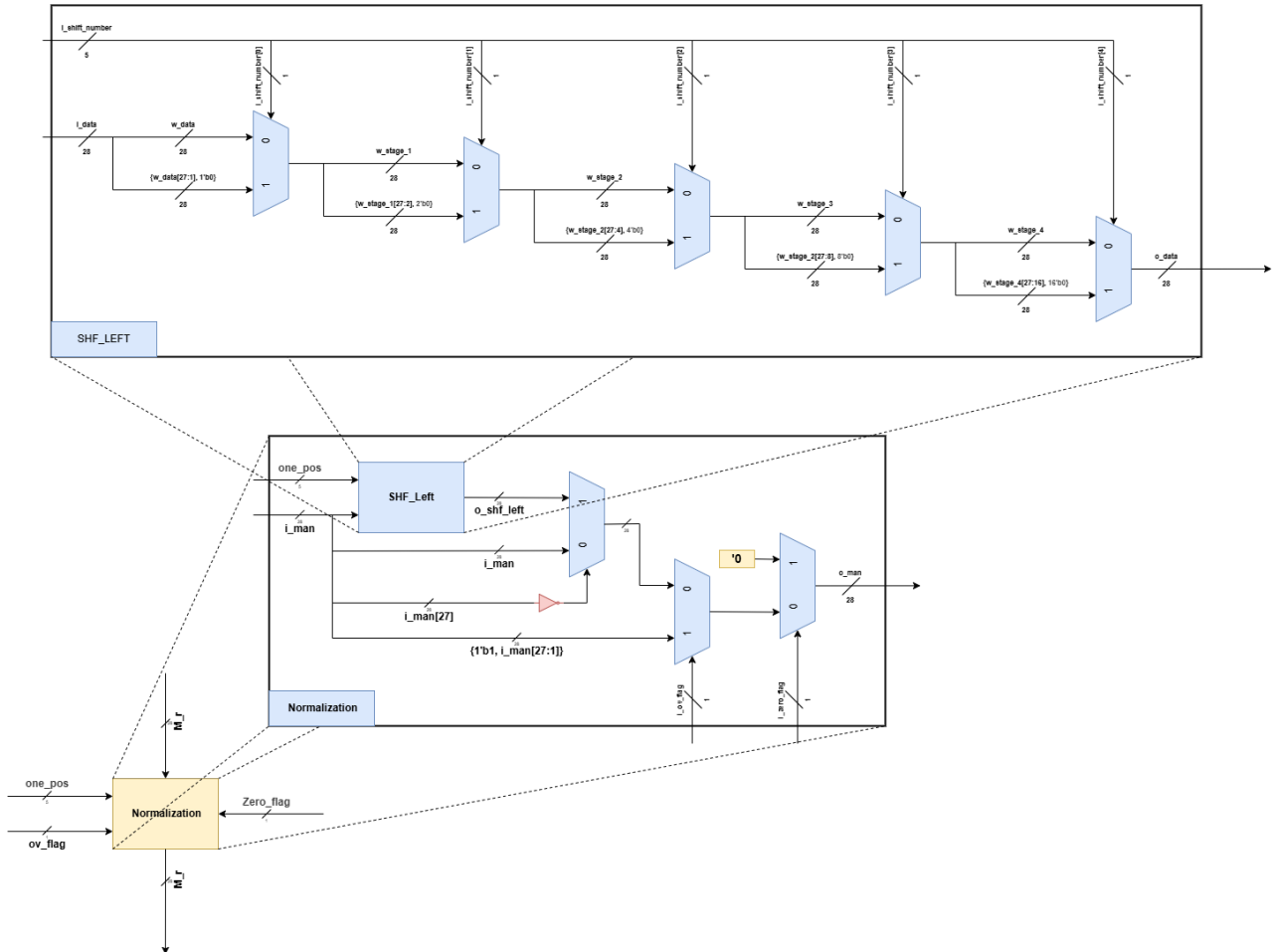
2.6 Bộ cộng trừ Mantissa



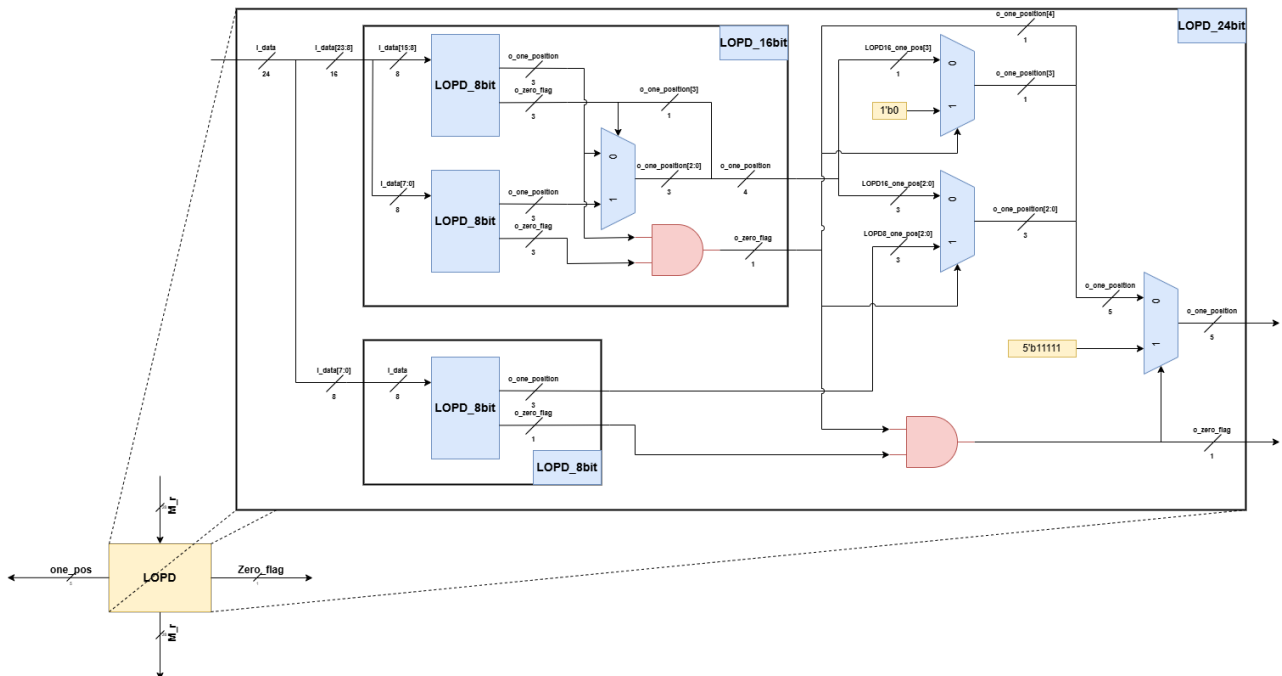
Tại bộ ALU_UNIT, nhóm em sử dụng một bộ cộng với 28-bit



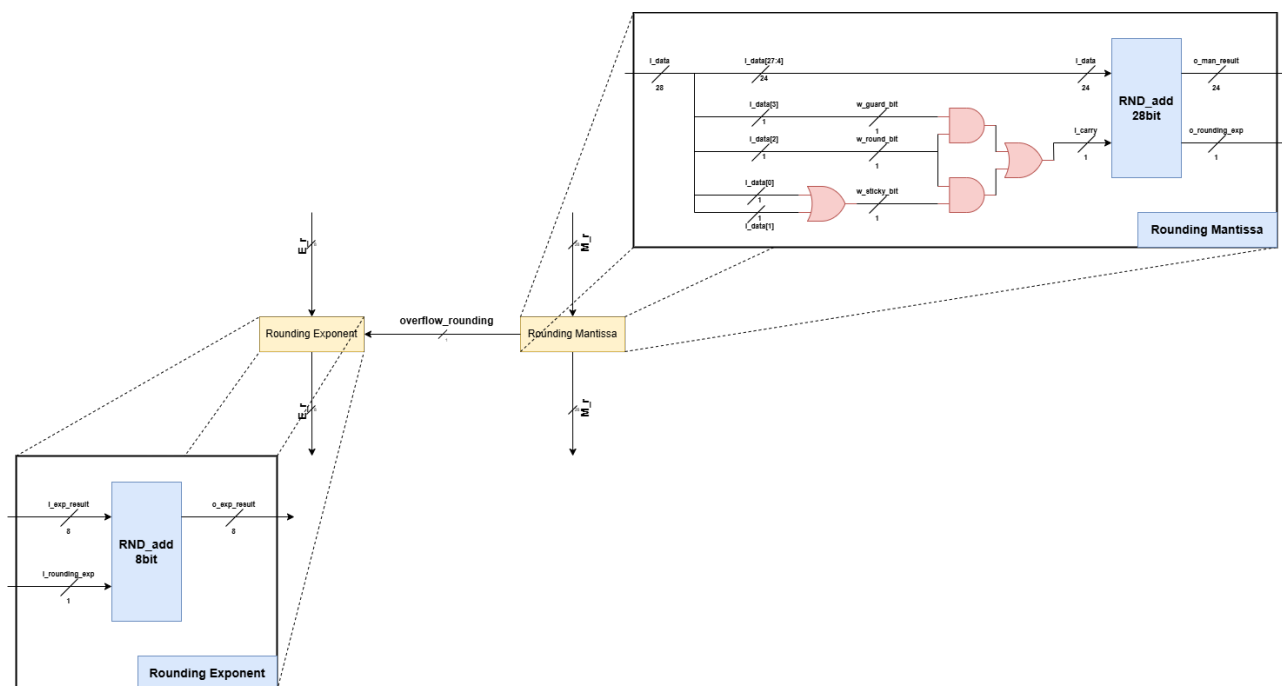
2.7 Bộ chuẩn hóa Mantissa



2.8 Bộ tìm vị trí bit 1 của MANTISSA sau khi chuẩn hóa



2.9 Bộ làm tròn cho giá trị Mantissa và Exponent



Ở đây bộ `RND_unit` là bộ cộng giá trị làm tròn cho dữ liệu đầu vào, với kiểu làm tròn là làm tròn gần nhất. Vì vậy, ở đây ta thấy ở đây chỉ lấy `i_data + i_carry` bởi vì chỉ là cộng thêm 1 hoặc không.

<pre>1 p = a ^ b; 2 g = a & b; 3 c[i] = g[i] (p[i] & c[i-1]) 4</pre>	$\Rightarrow$	<pre>1 p = a; 2 c[i] = g[i] (p[i] & c[i-1]) 3 4 s = p ^ c 5 c_o = &a & c[0] 6</pre>
---	---------------	---

Chapter 3. Simulation and Verification

Để thực hiện kiểm định thiết kế, nhóm em sử dụng kiểu dữ liệu `shortreal` của SystemVerilog để thực hiện làm một golden module như sau:

```

1  function automatic shortreal Cal_FPU_expected(
2      input logic          f_i_add_sub ,
3      input shortreal      f_i_32_a    ,
4      input shortreal      f_i_32_b
5  );
6      begin
7          return (f_i_add_sub) ? f_i_32_a - f_i_32_b : f_i_32_a + f_i_32_b;
8      end
9  endfunction

```

Listing 2: Hàm golden model để thực hiện test.

Nhóm em sử dụng các test theo sai số, với sai số nhóm em chọn là:

$$\text{Error_standard} = \frac{2.0^{-23}}{1.0} \times 100 \approx 11.9209 \mu\%$$

Có nghĩa là sai ở bit LSB của Mantissa.

Tiếp theo, ta thực hiện xây dựng hàm tính sai số giữa kết quả của golden model và kết quả từ bộ DUT,

```

1  function automatic shortreal Error_actual(
2      input shortreal      f_i_32_s    ,
3      input shortreal      f_i_32_e
4  );
5      logic [31:0] f_t_32_s, f_t_32_e;
6      logic is_E_one_S, is_E_one_E;
7      logic is_M_zero_S, is_M_zero_E;
8      logic is_INF_S, is_INF_E;
9      logic is_NAN_S, is_NAN_E;
10
11     begin
12         f_t_32_s = REAL_TO_HEX(f_i_32_s);
13         f_t_32_e = REAL_TO_HEX(f_i_32_e);
14         is_E_one_S = &f_t_32_s[30:23];
15         is_M_zero_S = ~|f_t_32_s[22:0];
16         is_INF_S = is_E_one_S & is_M_zero_S;
17         is_NAN_S = is_E_one_S & ~is_M_zero_S;
18         is_E_one_E = &f_t_32_e[30:23];
19         is_M_zero_E = ~|f_t_32_e[22:0];
20         is_INF_E = is_E_one_E & is_M_zero_E;
21         is_NAN_E = is_E_one_E & ~is_M_zero_E;
22
23         if(is_NAN_S) begin
24             Error_actual = (is_NAN_E) ? 0.0 : 100.0;
25         end else if(is_NAN_E) begin
26             Error_actual = (is_NAN_S) ? 0.0 : 100.0;
27         end else if(is_INF_S || is_INF_E) begin
28             Error_actual = (f_t_32_s == f_t_32_e) ? 0.0 : 100;
29         end else if((f_i_32_e == 0.0) || (f_i_32_e == -0.0)) begin

```

3 SIMULATION AND VERIFICATION

```
30     Error_actual = ((f_i_32_s == 0.0) || (f_i_32_s == -0.0)) ? 0.0 : 100.0;
31     end else begin
32         Error_actual = ((ABS_value(f_i_32_e - f_i_32_s)) / ABS_value(f_i_32_e)) * 100.0;
33     end
34 end
35 endfunction
```

Listing 3: Hàm tính sai số giữa kết quả của DUT và kết quả của golden model để thực hiện test.

Với xử lý các trường hợp như sau:

- Nếu kết quả của DUT là một NaN-(Not A Number), hoặc kết quả của bộ golden model là NaN, và khi kiểm tra trường hợp có xuất hiện NaN thì không quan tâm đến dấu của phép tính, vậy ta có kết quả sẽ kiểm tra như sau:
 - Nếu cả hai đều là NaN thì $\text{Error_actual} = 0.0\%$
 - Còn nếu một kết quả là NaN còn kết quả còn lại không phải là NaN thì $\text{Error_actual} = 100.0\%$
- Nếu kết quả của DUT là một $\pm\infty$, hoặc kết quả của bộ golden model là $\pm\infty$, thì kiểm tra kết quả của bộ DUT và golden model có giống nhau không:
 - Nếu cả hai giống nhau thì $\text{Error_actual} = 0.0\%$
 - Còn nếu hai kết quả không giống nhau thì $\text{Error_actual} = 100.0\%$
- Còn các trường hợp còn lại thì thực hiện việc tính sai số như sau:

$$\text{Error_actual} = \frac{|f_i_32_e - f_i_32_s|}{|f_i_32_e|} \times 100.0$$

Từ đó ta có hàm test chính như sau:

```
1     initial begin
2         i_rst_n = 0;
3         i_add_sub      = 1'b0;
4         i_32_a          = 32'h0;
5         i_32_b          = 32'h0;
6         w_i_addr        = '0;
7         #100;
8         i_rst_n = 1;
9         #100;
10        TestCase_Display_result("ZERO", "(0.0 & 0.0)", 32'h00000000, 32'h00000000);
11        TestCase_Display_result("ZERO", "(0.0 & -0.0)", 32'h00000000, 32'h80000000);
12        TestCase_Display_result("ZERO", "(0.0 & -0.0)", 32'h4016A197, 32'h4016A197);
13        TestCase_Display_result("ZERO", "(0.0 & -0.0)", 32'h40AED834, 32'h40AED834);
14        TestCase_Display_result("INF", "(inf & inf)", 32'h7f800000, 32'h7f800000);
15        TestCase_Display_result("INF", "(-inf & -inf)", 32'hff800000, 32'hff800000);
```

3 SIMULATION AND VERIFICATION

```
16 TestCase_Display_result("INF", "(inf & -inf)", 32'hff800000, 32'h7f800000);
17 TestCase_Display_result("INF", "(inf & 0)", 32'h7f800000, 32'h00000000);
18 TestCase_Display_result("INF", "(-inf & 0)", 32'hff800000, 32'h00000000);
19 TestCase_Display_result("INF", "(inf & Number)", 32'h7f800000, 32'h40533333);
20 TestCase_Display_result("INF", "(-inf & Number)", 32'hff800000, 32'h40533333);
21 TestCase_Display_result("INF", "(inf & -Number)", 32'h7f800000, 32'hc00ccccd);
22 TestCase_Display_result("INF", "(-inf & -Number)", 32'hff800000, 32'hc00ccccd);
23 TestCase_Display_result("NaN", "(NaN & -Number)", 32'h7f800001, 32'hc00ccccd);
24 TestCase_Display_result("NaN", "(-NaN & -Number)", 32'hff800001, 32'hc00ccccd);
25 TestCase_Display_result("NaN", "(NaN & Number)", 32'hff800001, 32'h40533333);
26 TestCase_Display_result("NaN", "(-NaN & Number)", 32'h7f800001, 32'h40533333);
27 TestCase_Display_result("APPRO", "APPR INF", 32'h7f21616f, 32'h007fffff);
28 TestCase_Display_result("APPRO", "APPR INF", 32'h7f7fffff, 32'h00fffff);
29 TestCase_Display_result("APPRO", "APPR INF", 32'h7f7fffff, 32'h007fffff);
30 TestCase_Display_result("APPRO", "APPR ZERO", 32'h00fffff, 32'h007fffff);
31 TestCase_Display_result("APPRO", "APPR ZERO", 32'h00fffff, 32'h00fffff);
32 TestCase_Display_result("SIGN", "(-A + B)", 32'hc00ccccd, 32'h40533333);
33 TestCase_Display_result("SIGN", "TEST SIGN", 32'hc00ccccd, 32'hc0533333);
34 TestCase_Display_result("SIGN", "TEST SIGN", 32'hc00ccccd, 32'hc1b1999a);
35 TestCase_Display_result("PRE_NOR_EXP", "Overflow rounding", 32'h0cffffff, 32'h00f80000);
36 repeat (2**SIZE_ADDR) begin
37     TestCase_Display_result("Random", "Read data from ROM", w_o_data_rom_a, w_o_data_rom_b);
38     @(posedge i_clk);
39     #1;
40     w_i_addr = w_i_addr + 1;
41 end
42
43 Display_SummaryResult(test_count, test_pass);
44 #100;
45 $finish;
46 end
```

Listing 4: Hàm Testbench chính.

Kết quả của Testbench (Compile.log)

```
===== [ (0.0 & 0.0) ] =====
[ZERO][ADD]i_32_a=00000000 (0.00000000000000000000000000000000) + i_32_b=00000000 (0.00000000000000000000000000000000) | o_32_s=00000000
(0.00000000000000000000000000000000) | o_ov_flow=1, o_un_flow=1
=> PASS: expect=0.00000000000000000000000000000000 (00000000), dut=0.00000000000000000000000000000000 (00000000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[ZERO][ADD]i_32_a=00000000 (0.00000000000000000000000000000000) + i_32_b=00000000 (0.00000000000000000000000000000000) | o_32_s=00000000
(0.00000000000000000000000000000000) | o_ov_flow=1, o_un_flow=1
=> PASS: expect=0.00000000000000000000000000000000 (00000000), dut=0.00000000000000000000000000000000 (00000000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[ZERO][SUB]i_32_a=00000000 (0.00000000000000000000000000000000) - i_32_b=00000000 (0.00000000000000000000000000000000) | o_32_s=00000000
(0.00000000000000000000000000000000) | o_ov_flow=0, o_un_flow=1
=> PASS: expect=0.00000000000000000000000000000000 (00000000), dut=0.00000000000000000000000000000000 (00000000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[ZERO][SUB]i_32_a=00000000 (0.00000000000000000000000000000000) - i_32_b=00000000 (0.00000000000000000000000000000000) | o_32_s=00000000
(0.00000000000000000000000000000000) | o_ov_flow=0, o_un_flow=1
=> PASS: expect=0.00000000000000000000000000000000 (00000000), dut=0.00000000000000000000000000000000 (00000000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
...
===== [ (inf & inf) ] =====
[INF][ADD]i_32_a=7f800000 (inf) + i_32_b=7f800000 (inf) | o_32_s=7f800000 (inf) | o_ov_flow=1, o_un_flow=1
=> PASS: expect=inf (7f800000), dut=inf (7f800000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[INF][ADD]i_32_a=7f800000 (inf) + i_32_b=7f800000 (inf) | o_32_s=7f800000 (inf) | o_ov_flow=1, o_un_flow=1
=> PASS: expect=inf (7f800000), dut=inf (7f800000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[INF][SUB]i_32_a=7f800000 (inf) - i_32_b=7f800000 (inf) | o_32_s=7fc00000 (nan) | o_ov_flow=0, o_un_flow=1
=> PASS: expect=nan (7fc00000), dut=nan (7fc00000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[INF][SUB]i_32_a=7f800000 (inf) - i_32_b=7f800000 (inf) | o_32_s=7fc00000 (nan) | o_ov_flow=0, o_un_flow=1
=> PASS: expect=nan (7fc00000), dut=nan (7fc00000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
...
===== [ (-NaN & Number) ] =====
[NaN][ADD]i_32_a=7f800001 (nan) + i_32_b=40533333 (3.299999952316284179687500) | o_32_s=7fc00000 (nan) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=nan (7fc00001), dut=nan (7fc00000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[NaN][ADD]i_32_a=40533333 (3.299999952316284179687500) + i_32_b=7f800001 (nan) | o_32_s=7fc00000 (nan) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=nan (7fc00001), dut=nan (7fc00000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[NaN][SUB]i_32_a=7f800001 (nan) - i_32_b=40533333 (3.299999952316284179687500) | o_32_s=7fc00000 (nan) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=nan (7fc00001), dut=nan (7fc00000), rounding_error=0.00000000 % (exp_error = 0.00001192 %)
[NaN][SUB]i_32_a=40533333 (3.299999952316284179687500) - i_32_b=7f800001 (nan) | o_32_s=ffc00000 (-nan) | o_ov_flow=0, o_un_flow=0
```

3 SIMULATION AND VERIFICATION

```
=> PASS: expect=nan (7fc00001), dut=-nan (ffc00000), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
...
===== [ APPR INF ] =====
[APPRO][ADD]i_32_a=7f21616f (214511611464115339152897452993962573824.000000000000000000000000) + i_32_b=007fffff (0.000000000000000000000000) |
o_32_s=7f21616f (214511611464115339152897452993962573824.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=214511611464115339152897452993962573824.000000000000000000000000 (7f21616f), dut
=214511611464115339152897452993962573824.000000000000000000000000 (7f21616f), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
[APPRO][ADD]i_32_a=007fffff (0.000000000000000000000000) + i_32_b=7f21616f (214511611464115339152897452993962573824.0000000000000000000000) |
o_32_s=7f21616f (214511611464115339152897452993962573824.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=214511611464115339152897452993962573824.000000000000000000000000 (7f21616f), dut
=214511611464115339152897452993962573824.000000000000000000000000 (7f21616f), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
[APPRO][SUB]i_32_a=7f21616f (214511611464115339152897452993962573824.000000000000000000000000) - i_32_b=007fffff (0.000000000000000000000000) |
o_32_s=7f21616f (214511611464115339152897452993962573824.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=214511611464115339152897452993962573824.000000000000000000000000 (7f21616f), dut
=214511611464115339152897452993962573824.000000000000000000000000 (7f21616f), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
[APPRO][SUB]i_32_a=007fffff (0.000000000000000000000000) - i_32_b=7f21616f (214511611464115339152897452993962573824.0000000000000000000000) |
o_32_s=ff21616f (~214511611464115339152897452993962573824.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=-214511611464115339152897452993962573824.000000000000000000000000 (ff21616f), dut
=-214511611464115339152897452993962573824.000000000000000000000000 (ff21616f), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
===== [ Overflow rounding ] =====
[PRE_NOR_EXP][ADD]i_32_a=0cfffff (0.000000000000000000000000) + i_32_b=00f80000 (0.000000000000000000000000) | o_32_s=0d000000
(0.000000000000000000000000) | o_ov_flow=1, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (0d000000), dut=0.000000000000000000000000 (0d000000), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
[PRE_NOR_EXP][ADD]i_32_a=00f80000 (0.000000000000000000000000) + i_32_b=0cfffff (0.000000000000000000000000) | o_32_s=0d000000
(0.000000000000000000000000) | o_ov_flow=1, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (0d000000), dut=0.000000000000000000000000 (0d000000), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
[PRE_NOR_EXP][SUB]i_32_a=0cfffff (0.000000000000000000000000) - i_32_b=00f80000 (0.000000000000000000000000) | o_32_s=0cfffff
(0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=0.000000000000000000000000 (0cfffff), dut=0.000000000000000000000000 (0cfffff), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
[PRE_NOR_EXP][SUB]i_32_a=00f80000 (0.000000000000000000000000) - i_32_b=0cfffff (0.000000000000000000000000) | o_32_s=8cfffff
(~0.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=-0.000000000000000000000000 (8cfffff), dut=-0.000000000000000000000000 (8cfffff), rounding_error=0.0000000 % (exp_error = 0.00001192 %)
...
===== [ Read data from ROM ] =====
[Random][ADD]i_32_a=d09f76d2 (~21402914816.000000000000000000000000) + i_32_b=501fb38a (10717374464.000000000000000000000000) | o_32_s=d01f3a1a
(~10685540352.000000000000000000000000) | o_ov_flow=0, o_un_flow=1
=> PASS: expect=-10685540352.000000000000000000000000 (d01f3a1a), dut=-10685540352.000000000000000000000000 (d01f3a1a), rounding_error=0.0000000 % (
exp_error = 0.00001192 %)
[Random][ADD]i_32_a=501fb38a (10717374464.000000000000000000000000) + i_32_b=d09f76d2 (~21402914816.000000000000000000000000) | o_32_s=d01f3a1a
(~10685540352.000000000000000000000000) | o_ov_flow=0, o_un_flow=1
=> PASS: expect=-10685540352.000000000000000000000000 (d01f3a1a), dut=-10685540352.000000000000000000000000 (d01f3a1a), rounding_error=0.0000000 % (
exp_error = 0.00001192 %)
[Random][SUB]i_32_a=d09f76d2 (~21402914816.000000000000000000000000) - i_32_b=501fb38a (10717374464.000000000000000000000000) | o_32_s=d0ef5097
(~32120289280.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=-32120289280.000000000000000000000000 (d0ef5097), dut=-32120289280.000000000000000000000000 (d0ef5097), rounding_error=0.0000000 % (
exp_error = 0.00001192 %)
[Random][SUB]i_32_a=501fb38a (10717374464.000000000000000000000000) - i_32_b=d09f76d2 (~21402914816.000000000000000000000000) | o_32_s=50ef5097
(32120289280.000000000000000000000000) | o_ov_flow=0, o_un_flow=0
=> PASS: expect=32120289280.000000000000000000000000 (50ef5097), dut=32120289280.000000000000000000000000 (50ef5097), rounding_error=0.0000000 % (
exp_error = 0.00001192 %)

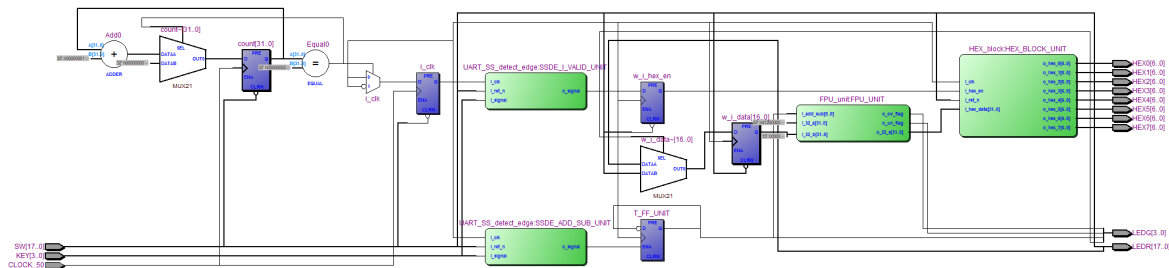
=====
===== TEST SUMMARY =====
Total test cases: 8296
Passed : 8296
Failed : 0
Pass rate : 100.00%
=====

Simulation complete via $finish(1) at time 207371 NS + 0
../Topmodule/tb_FPU_unit.sv:234 $finish;
xcelium> exit
T00L: xrun(64)
```

Listing 5: Kết quả của Testbench.

Chapter 4. Implement on FPGA

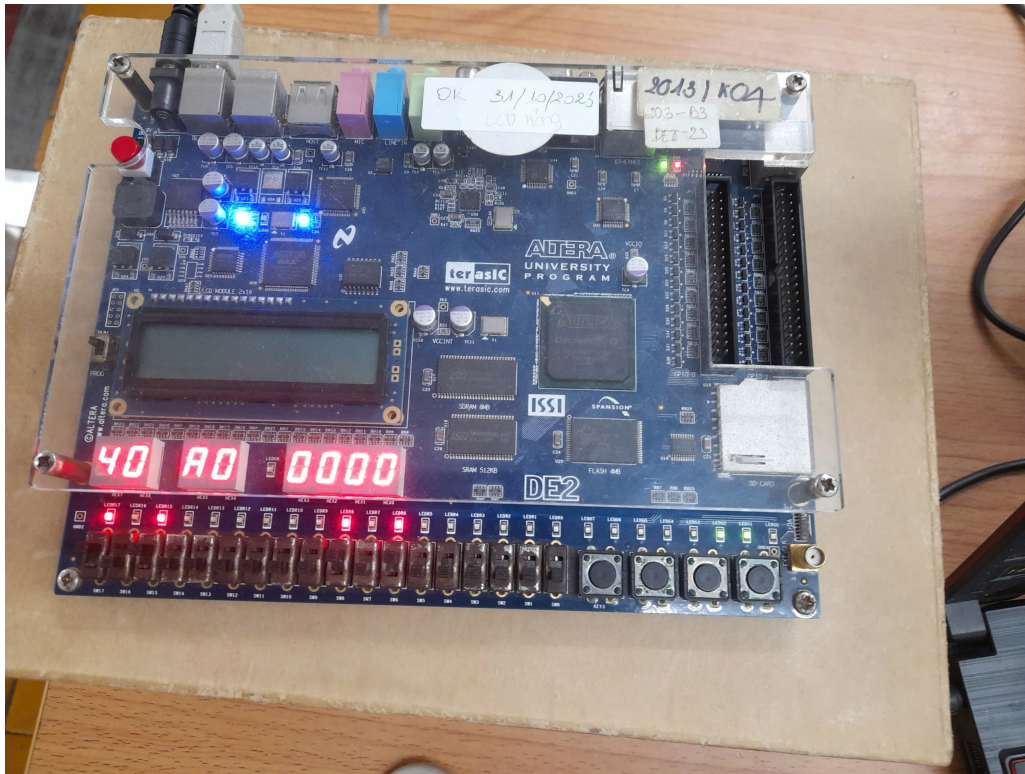
Sử dụng DE2 để thực hiện kiểm chứng bộ FPU, tổng quan thiết kế của bộ khi thực hiện như sau:



Hình 4.1: RTL Viewer của bộ FPU.

Với việc cố định giá trị $i_32_a = 32'h41200000$ (tương ứng với 10.0). Kết quả sau khi đổ kit như sau:

4 IMPLEMENT ON FPGA



Hình 4.2: Kết quả khi thực hiện trên kit DE2.

Chapter 5. Synthesis my design

Nhóm em sử dụng Standard Cells là:

+ slow_vdd1v2_basicCells_lvt.lib

+ slow_vdd1v2_basicCells_lvt.v

Tiến hành sử dụng file .tcl sau để thực hiện trên Genus:

```
1 #####
2 #
3 # EE3213_04 (cre: Khai Pham)
4 # Digital system design and Verification
5 #
6 #####
7
8 set top_module FPU_unit
9 set_db library [list ../../slow_vdd1v2_basicCells_lvt.lib ]
10 #HDL files read#
11 read_hdl -sv -f ./flist.f
12 elaborate
13 set_top_module ${top_module}
14 write_hdl > ./Genus/${top_module}_elab.v
15 check_design -all
16 read_sdc ./constraint.sdc
17 syn_generic
18 write_hdl > ./Genus/${top_module}_generic.v
19 syn_map
20 write_hdl > ./Genus/${top_module}_tech_map.v
21 remove_assigns_without_optimization -verbose
22 syn_opt
23 report timing -lint
24 write_sdf -edges check_edge -setuphold "split" -recrem split > ./Genus/${top_module}.sdf
25 report timing -max_paths 10 > ./Genus/report/${top_module}.timing.rpt
26 report hierarchy > ./Genus/report/${top_module}.hier.rpt
27 report gates > ./Genus/report/${top_module}.gates.rpt
28 report datapath > ./Genus/report/${top_module}.datapath.rpt
29 report qor > ./Genus/report/${top_module}.qor.rpt
30 report area > ./Genus/report/${top_module}.area.rpt
31 report power > ./Genus/report/${top_module}.power.rpt
32 gui_show
```

Listing 6: genus_synthesis.tcl

Với file **constraint.sdc** như sau:

```
1 set_max_delay 2.0 -from [all_inputs] -to [all_outputs]
2 set_input_delay 0.1 -clock [get_clocks *] [all_inputs]
3 set_output_delay 0.1 -clock [get_clocks *] [all_outputs]
4 set timing_report_unconstrained true
```

Listing 7: File constraint.sdc

Kết quả của **timing.rpt** như sau:

Path 1: MET (0 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (R) o_32_s[19]

Capture	Launch	
Path Delay	:+ 2000	-
Arrival	:= 2000	

Required Time	:= 2000
Data Path	:- 2000
Slack	:= 0

Path 2: MET (0 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (F) o_32_s[20]

Capture	Launch	
Path Delay	:+ 2000	-
Arrival	:= 2000	

Required Time	:= 2000
Data Path	:- 2000
Slack	:= 0

Path 3: MET (0 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (F) o_32_s[17]

Capture	Launch	
Path Delay	:+ 2000	-
Arrival	:= 2000	

Required Time	:= 2000
Data Path	:- 2000
Slack	:= 0

Path 4: MET (0 ps) Path Delay Check
Startpoint: (R) i_32_a[27]
Endpoint: (R) o_32_s[25]

Capture	Launch	
Path Delay	:+ 2000	-
Arrival	:= 2000	

Required Time	:= 2000
Data Path	:- 2000
Slack	:= 0

Path 5: MET (0 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (R) o_32_s[29]

Capture	Launch	
Path Delay	:+ 2000	-
Arrival	:= 2000	

Required Time	:= 2000
Data Path	:- 2000
Slack	:= 0

Path 6: MET (1 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (R) o_32_s[28]

Capture	Launch	
Path Delay	:+ 2000	-
Arrival	:= 2000	

Required Time	:= 2000
Data Path	:- 1999
Slack	:= 1

Path 7: MET (1 ps) Path Delay Check
Startpoint: (R) i_32_a[27]
Endpoint: (R) o_32_s[30]

Capture	Launch	
Path Delay	:+	2000 -
Arrival	:=	2000

Required Time	:=	2000
Data Path	:-	1999
Slack	:=	1

Path 8: MET (5 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (R) o_32_s[21]

Capture	Launch	
Path Delay	:+	2000 -
Arrival	:=	2000

Required Time	:=	2000
Data Path	:-	1995
Slack	:=	5

Path 9: MET (8 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (R) o_32_s[14]

Capture	Launch	
Path Delay	:+	2000 -
Arrival	:=	2000

Required Time	:=	2000
Data Path	:-	1992
Slack	:=	8

Path 10: MET (10 ps) Path Delay Check
Startpoint: (R) i_32_b[29]
Endpoint: (R) o_32_s[16]

Capture	Launch	
Path Delay	:+	2000 -
Arrival	:=	2000

Required Time	:=	2000
Data Path	:-	1990
Slack	:=	10

